

Docker Container Topology Visualizer

Project Requirements and Problem Analysis

Name: Ashmit Tyagi

Roll no. : E23CSEU1117

Report 1: Initial Requirements Report

1. Problem Understanding

The main problem I am trying to solve is that when a project has multiple Docker containers using Docker Compose, it can become difficult to understand how everything is connected. Usually, I have to open the docker-compose.yml file and check it manually to see which service depends on which one, what ports are being used, which networks are connected and what volumes are shared. For a small project this is fine, but when there are many services the file can become confusing.

So the idea is to make a simple tool where the user gives the docker-compose.yml file and the tool reads it and creates a visual diagram of the setup.

Basically the flow will be:

Input: docker-compose.yml file

Processing: Read the file and find services, ports, networks, volumes and dependencies

Output: A visual diagram showing the containers/services and their connections

2. Stakeholders

- Developers – They can use the tool to understand a project's Docker setup without going through the whole Compose file.
- DevOps engineers – They can use it to quickly check things like container connections, ports and networks, especially while troubleshooting.
- Project managers – They may not need all the technical details but a diagram can help them understand the overall system.
- Students or other users – Anyone working with Docker Compose can use it to get a quick visual idea of the setup.

3. Applications/Tools Related to the Project

- Docker / Docker Compose – Used for creating and running the containers that my tool will analyze.
- Python – I am planning to use Python for processing the Compose file.
- PyYAML – This will be used to read and parse the YAML file.
- Flask – I can use Flask for the backend and to connect the frontend with the Python code.
- Cytoscape.js – I am planning to use this for creating the interactive graph.
- HTML/CSS/JavaScript – These will be used for the frontend.
- Git/GitHub – Used for storing the project and tracking changes to the code.

If I add the live status feature, I can use Docker SDK for Python to get information from the running Docker containers.

4. Functional and Non-Functional Requirements

Functional requirements:

- The tool should allow the user to upload or provide a docker-compose.yml file.
- It should read the services present in the file.
- It should find the ports, networks and volumes being used.
- It should identify basic dependencies between services, such as depends_on.
- It should convert this information into a visual graph.
- The user should be able to click on a service and see some basic details about it.
- The diagram should be exportable or savable.

Non-functional requirements:

- The tool should be simple and easy to use.
- The diagram should be reasonably clear even when there are many services.
- The application should not crash if the Compose file has some errors. It should show an understandable error message instead.

- It should work on common operating systems like Windows, Linux and macOS.
- Sensitive information such as passwords, tokens or API keys should not be displayed.
- The tool should be fast enough for normal-sized Compose files.

5. System Components

1. File Input – The user provides the Docker Compose file.
2. Parser – Reads the YAML file and extracts the information I need.
3. Graph Builder – Takes the extracted information and creates nodes and connections.
4. Visualization – Displays the nodes and connections as an interactive diagram.
5. Details – Shows extra information when the user selects a service.
6. Export – Allows the user to save the generated diagram.

The flow will be:

Compose file → Parser → Extract information → Graph Builder → Visual diagram → Export

6. Technology Stack

The technologies to be used are:

- Python – Main programming language
- PyYAML – For reading the Compose YAML file
- Flask – Backend/web server
- Cytoscape.js – For displaying the interactive graph
- HTML/CSS/JavaScript – Frontend
- Docker SDK for Python – Only for the live container status feature
- Git/GitHub – Version control and tracking changes

Some of these tools might change if I find an easier option while actually developing the project.

7. Assumptions and Modifications

- Only one Compose file will be analyzed at a time.
- The main feature will be the static visualization of the Compose file.
- Live container monitoring is an extra feature and is not necessary for the basic version.
- I will not display sensitive information such as passwords, tokens or API keys from the Compose file.
- I am assuming that the user already has some basic knowledge of Docker and Docker Compose.
- Full Docker container management is not part of this project. Kubernetes support is also out of scope for now, but it could be considered as a future improvement.

GenAI Review and Analysis

Prompt given to Claude (GenAI)

“I'm working on a requirements analysis report for a college project called "Docker Container Topology Visualizer" a tool that analyzes a Docker Compose file and visualizes containers, networks, volumes, ports, and service dependencies.

Below is my initial requirements report, covering: problem understanding, stakeholders, related applications, functional/non-functional requirements, system components, technology stack, and assumptions. Please review it critically and

1. Point out anything missing, unclear, or weak in each section.
2. Suggest additional functional or non-functional requirements I may have overlooked.
3. Suggest any additional stakeholders or system components I might have missed.
4. Comment on whether the technology stack choices make sense for this project, and suggest alternatives if relevant.
5. Flag any assumptions that seem risky or unrealistic.
6. Give me an improved/polished version of the report incorporating your suggestions.”

Here is my report.

Summary of what Claude suggested

The response was a lot more detailed than the original draft. Some of the main things it added were:

- Support for override compose files and .env variable interpolation
- Using `yaml.safe_load()` instead of plain `yaml.load()` for security reasons
- Being upfront that secret-hiding (passwords/tokens) is best-effort and pattern based, not a guaranteed feature
- A proper comparison with existing tools like Weave Scope, Portainer, dcv and Lens
- Processing everything locally instead of uploading files anywhere
- Full schema validation using jsonschema and the official Compose Specification
- Handling extends: references between compose files
- Testing the parser against YAML anchors and aliases
- A specific performance target (50 services in under 3 seconds)
- Adding instructors/evaluators as a stakeholder

Overall, Claude gave some genuinely good security and structuring ideas I probably wouldn't have thought of on my own, especially around YAML safety. But it also added a bunch of stuff that felt more like it was trying to sound impressive/thorough rather than being realistic for the scope of what I'm actually building. Tried to only keep the parts I could actually commit to and defend if asked about them.

Report 2: Polished Requirements Report

1. Problem Understanding

The basic problem is the same as the first report. When a project has a lot of containers running through Docker Compose, it can be difficult to understand how everything is connected just by looking at the `docker-compose.yml` file. I have to check the services, ports, networks, volumes and dependencies manually, which can get confusing as the project becomes bigger.

So the main idea of my project is to take the Compose file and convert the information into an interactive visual diagram. Instead of reading the whole YAML file, the user can look at the diagram and get an idea of how the application is structured.

Basically the flow will be:

Input: `docker-compose.yml` file

Processing: Read the file and find services, ports, networks, volumes and dependencies

Output: A visual diagram showing the containers/services and their connections

2. Stakeholders

- Developers – They can quickly understand the Docker setup of a project without going through the entire YAML file.
- DevOps engineers – They can use the diagram to check things like networks, ports and service dependencies, especially when trying to find configuration problems.
- Project managers – They may not need all the technical details, but a simple diagram can help them understand the overall architecture.
- Students and other users – It can also be useful for students who are learning Docker Compose and want to understand it visually.

3. Applications Used in the Project

- Docker / Docker Compose – Used for creating and running the containers that my tool will analyze.
- Python – I am planning to use Python for processing the Compose file.
- PyYAML – This will be used to read and parse the YAML file.
- Flask – I can use Flask for the backend and to connect the frontend with the Python code.
- Cytoscape.js – I am planning to use this for creating the interactive graph.
- HTML/CSS/JavaScript – These will be used for the frontend.
- Git/GitHub – Used for storing the project and tracking changes to the code.

If I add the live status feature, I can use Docker SDK for Python to get information from the running Docker containers.

4. Functional and Non-Functional Requirements

Functional requirements:

- The user should be able to provide a docker-compose.yml file.
- It should find the services, ports, networks, volumes and dependencies in the Compose file.
- It should identify basic connections between services using things like depends_on and shared networks.

- If the YAML file is invalid, the application should show an understandable error instead of crashing.
- When a service is selected, some basic information such as image, ports, volumes and dependencies should be shown.
- The user should also be able to export or save the generated diagram.

Non-functional requirements:

- The tool should be simple enough that a user can understand how to use it without needing a lot of instructions.
- The diagram should remain reasonably readable when there are more services. If it becomes too crowded, I can add some basic filtering options.
- It should be reasonably fast for normal Compose projects and should not feel slow while generating the diagram.
- The Compose file should be processed locally and should not be uploaded to some external server.
- The code should be divided into separate parts such as the parser, graph builder and visualizer so that adding features later is easier.
- The YAML file should always be parsed safely (using `safe_load`) so that a malicious or broken file cannot cause unexpected behavior.
- Sensitive values such as passwords, tokens or API keys should not be displayed

5. System Components

The project will be divided into a few main parts:

1. File Input – takes the Docker Compose file.
2. Parser – reads the final Compose configuration and extracts the information I need.
3. Graph Builder – converts the extracted information into nodes and connections.
4. Visualization – displays the graph in the browser and allows basic interaction.
5. Details Panel – shows information about a selected service.
6. Export – allows the user to save the generated diagram.
7. Error Handler – handles invalid files and displays useful error messages.

6. Technology Stack

The technologies to be used are:

- Programming Language: Python
- Backend: Flask
- Configuration/File Parsing: PyYAML
- Frontend: HTML, CSS, JavaScript
- Visualization: Cytoscape.js
- Container Technology: Docker and Docker Compose
- Version Control: Git and GitHub
- Optional: Docker SDK for Python for checking the status of running containers

Some of these tools might change if I find an easier option while actually developing the project.

7. Assumptions and Modifications

- For this version, I am keeping the scope limited so that the project is actually manageable.
- The main input will be one Docker Compose file.
- The project will mainly focus on static analysis of the Compose file. Live container monitoring is still an optional feature if I have enough time.
- Also know that YAML supports things like anchors and aliases, but I am not specifically focusing on advanced cases like these in the first version. If they cause problems, I will mention them as a limitation.
- Kubernetes and Docker Swarm support are outside the scope of this project.
- Features for actually managing containers, such as starting, stopping or restarting them, are also outside the scope.
- Finally, I am assuming that the person using the tool already knows the basic concepts of Docker Compose. The tool is meant to make the structure easier to understand, not to teach Docker from the beginning.

